CampusIQ - Day 3 Notes

Name: Fahad Ahmed **Date:** _____

Topic: Many tables, FOREIGN KEY, JOIN and GROUP BY

1. Why More Than One Table?

- Putting everything in **one big table is wrong.** We would repeat the same data again and again, and one change would need many edits.
- So we make **separate tables for separate things:** students, marks and attendance.
- Then we **connect the tables** using the roll_no column.
- Benefit: no repeated data, easy to update, clean and safe.

Example: a student has 3 subjects. In one big table the name and branch would be written 3 times. In separate tables the name is written only once, in students.

2. Our 3 Tables

students (from Day 2). **roll_no is the PRIMARY KEY.**

roll_no	name	branch	cgpa
21AI01	Ravi	AIML	8.4
21AI02	Asha	AIML	7.2
...	...	...	...

marks (12 rows in total, 3 subjects for each student):

roll_no	subject	internal	external
21AI01	ML	34	52
21AI01	DBMS	30	45
21AI01	OS	28	40

attendance (12 rows in total):

roll_no	subject	attended	total
21AI01	ML	50	60
21AI01	DBMS	47	60
21AI01	OS	45	60

One student has many rows in marks and attendance (one for each subject). roll_no connects them to the students table.

3. FOREIGN KEY

- A **PRIMARY KEY** is the unique ID of a row inside its own table (roll_no in students).
- A **FOREIGN KEY** is a column that points to the PRIMARY KEY of another table (roll_no in marks points to roll_no in students).

In SQL we write it with **REFERENCES**:

```
roll_no TEXT REFERENCES students(roll_no)
```

- **Meaning**: marks can only have a roll_no that already exists in students. A roll_no that is not in students is not allowed.
- roll_no repeats in marks (3 times for each student). That is fine, because it is a foreign key, not a primary key.

- **Order matters**: create and fill students first (the parent), then marks and attendance (the children).

Test: try to insert a roll_no that is not in students. You will get this error:

```
violates foreign key constraint
```

This error protects our data from wrong entries.

4. Creating marks and attendance

marks table:

```
CREATE TABLE marks (
  roll_no TEXT REFERENCES students(roll_no),
  subject TEXT,
  internal INT,
  external INT
);
```

attendance table:

```
CREATE TABLE attendance (
  roll_no TEXT REFERENCES students(roll_no),
  subject TEXT,
  attended INT,
  total INT
);
```

- **INT** = whole numbers (no decimals). Good for marks and class counts.
- **internal** and **external** = two parts of the marks of a subject. Subject total = internal + external.
- **attended** = classes attended. **total** = total classes held.

- After INSERT the message must be **INSERT 0 12** (12 rows added) for each table.
- Run the CREATE and INSERT only once. A second run gives the error: already exists.

Check: run SELECT COUNT(*) FROM marks; and the same for attendance. Both must give 12.

5. JOIN - Query 1

JOIN = look at two tables together. It matches rows where the roll_no is the same in both tables.

```
SELECT s.name, m.subject,
       m.internal + m.external AS total
FROM students s JOIN marks m
  ON s.roll_no = m.roll_no
ORDER BY s.name;
```

- `students s` and `marks m` = short names (aliases). Now we can write `s.name` instead of `students.name`.
- **JOIN** with `ON s.roll_no = m.roll_no` = join the rows where the roll_no matches.
- `m.internal + m.external AS total` = add the two marks and call the new column total.
- `ORDER BY s.name` = sort by student name (A to Z).
- **Result: 12 rows** (4 students x 3 subjects). The subject order inside a student may differ on your screen.

Result:

name	subject	total
Asha	ML	63
Asha	DBMS	75
Asha	OS	52
Imran	ML	95
Imran	DBMS	86
Imran	OS	90
Ravi	ML	86
Ravi	DBMS	75
Ravi	OS	68
Zoya	ML	48
Zoya	DBMS	57
Zoya	OS	43

6. GROUP BY and SUM - Query 2

GROUP BY = put all rows of one student into one group and show one result line for each group. Then **SUM** adds up the numbers inside each group.

```
SELECT s.name,
       SUM(m.internal + m.external) AS total_marks
FROM students s JOIN marks m
  ON s.roll_no = m.roll_no
GROUP BY s.name
ORDER BY total_marks DESC;
```

- **Steps:** 1) JOIN the tables. 2) GROUP BY puts the 3 rows of each student together. 3) SUM adds the 3 subject totals. 4)

ORDER BY … DESC puts the highest on top.

Working: Imran 95 + 86 + 90 = 271. Ravi 86 + 75 + 68 = 229. Asha 63 + 75 + 52 = 190. Zoya 48 + 57 + 43 = 148.

Result:

name	total_marks
Imran	271
Ravi	229
Asha	190
Zoya	148

Rule: every column in SELECT must be either in GROUP BY or inside SUM, AVG, COUNT, MAX or MIN.

7. Attendance Percentage - Query 3

Aim: find students whose attendance is below 75% in any subject.

```
SELECT s.name, a.subject,
  ROUND(a.attended * 100.0 / a.total, 1) AS pct
FROM students s JOIN attendance a
  ON s.roll_no = a.roll_no
WHERE a.attended * 100.0 / a.total < 75
ORDER BY pct;
```

- **Percentage = attended x 100 / total.**
- **Why `100.0` and not `100`?** Because attended and total are INT. Whole numbers divided by whole numbers cut off the decimal part. Writing `100.0` makes the answer a proper decimal.
- `ROUND(value, 1)` = round to 1 digit after the decimal point.

- **WHERE** with < 75 = Keep only the rows below 75%.
- **We write the whole formula again in WHERE.** The short name pct cannot be used inside WHERE, but it can be used in ORDER BY.

Result: 6 rows

name	subject	pct
Zoya	OS	55.0
Asha	OS	60.0
Zoya	ML	63.3
Asha	DBMS	66.7
Asha	ML	70.0
Zoya	DBMS	73.3

Ravi in OS has exactly 75.0%. The condition is < 75, not <=, so he is not in the list.

8. Extra Practice

Try these yourself. The expected result is written beside each one.

```
SELECT * FROM marks WHERE subject = 'ML';
```

Result: 4 rows, one ML row for each student. Text in WHERE needs single quotes.

```
SELECT subject, AVG(external)
FROM marks GROUP BY subject;
```

Average external marks of each subject: **ML 43.75, DBMS 43, OS 37.5.** The order of rows may differ.

```
SELECT s.name, a.subject,
  ROUND(a.attended * 100.0 / a.total, 1) AS pct
FROM students s JOIN attendance a
  ON s.roll_no = a.roll_no
WHERE a.attended * 100.0 / a.total >= 90
ORDER BY pct DESC;
```

Attendance of 90% or more: **Imran ML 95.0, Imran DBMS 91.7, Imran OS 90.0.**

9. Save the SQL and Push

1. In VS Code open the db folder inside backend. Make a new file and paste all the SQL (CREATE, INSERT and the 3 queries). Save with Ctrl+S. File name:

```
backend/db/day3_marks_attendance.sql
```

2. Open the terminal in the campusiq folder (the top folder) and run one by one:

```
git pull
git add .
git commit -m "day 3: marks, attendance, joins"
git push
```

3. Open the GitHub repo and check that the new file is inside backend and then db.
4. Save these notes with this name in the notes folder and push the same way:

```
notes/fahad/day03.pdf
```

10. Common Errors and Fixes

- **relation "students" does not exist** - the Day 2 table is missing. Run the Day 2 CREATE and INSERT first.
- **relation "marks" already exists** - the table is already made. Do not run CREATE again.
- **violates foreign key constraint** - that roll_no is not in the students table. Check the roll_no spelling.
- **column reference "roll_no" is ambiguous** - both tables have roll_no. Write `s.roll_no` or `m.roll_no`.
- **must appear in the GROUP BY clause or be used in an aggregate function** - a column in SELECT is neither in GROUP BY nor inside SUM, AVG, COUNT, MAX or MIN.
- **column "pct" does not exist** - the short name was used in WHERE. Write the full formula in WHERE.
- **Only the last result shows** - run one query at a time. Clear the editor with Ctrl+A and Delete.
- **Cannot drop table students** - other tables depend on it. Drop marks and attendance first. (DROP deletes the data. Use it carefully.)

11. Revision Questions

Q1. Why do we use separate tables?

To avoid repeating the same data, to update data easily, and to keep each table clean and simple.

Q2. Difference between PRIMARY KEY and FOREIGN KEY?

A **primary key** is the unique ID of a row in its own table. A **foreign key** is a column that points to the primary key

of another table.

Q3. What does JOIN do?

It joins rows of two tables together, using a matching column. Here we match roll_no in students and marks.

Q4. What does GROUP BY do?

It puts the rows with the same value (like one student) into one group, so that SUM, AVG or COUNT can be calculated for each group.

Q5. Why do we write `100.0` and not `100` in the percentage?

Because INT divided by INT drops the decimal part. `100.0` gives a correct decimal answer.

Q6. Why is Ravi missing in Query 3?

His OS attendance is exactly 75.0, and the condition is below 75 (< 75), so it does not include him.

12. Day 3 Checklist

- ☐ Pulled the latest code with git pull
- ☐ marks and attendance tables made (12 rows each)
- ☐ Query 1 gave 12 rows
- ☐ Query 2: Imran 271, Ravi 229, Asha 190, Zoya 148
- ☐ Query 3 gave 6 rows, Ravi is not in the list
- ☐ SQL file saved in the backend and db folder
- ☐ Pushed to GitHub and the file is visible
- ☐ Notes saved as day03 in my notes folder

Important: Tables + FOREIGN KEY + JOIN = clean data.